Perturbation Self-Consistency: A Cheap Black-Box Baseline for Predicting Reasoning Robustness, and Where It Fails

Alan De Bortolo
Design Master Solutions LLC
info@designmastersolutions.com

Abstract

The AIMO Interpretability Challenge asks whether a math-reasoning model's answer to a problem survives perturbations that leave the deductive chain intact. We submit a training-free, black-box method that measures this property directly at test time: the evaluated checkpoint answers the original problem and three rule-based perturbations (letter-swap typos, an irrelevant distractor sentence, character renaming) with greedy decoding, and the case is called robust only if every answer agrees. The method needs no labels, no weights access beyond generation and no learned readout, and it runs inside the 3600 s budget with full coverage. On the validation leaderboard it reaches 0.619 accuracy on the Main Model Track (submission 1019467) and 0.571 on the Small Model Track (submission 1019144), against 0.500 for the always-`False` predictor on the Small track (submission 1019672). We report this as a partly negative result: the gain over the majority class is modest, the always-`False` predictor reaches 0.68 on the public sample, and an analysis of that sample shows that robustness labels depend first on whether the model solves the original problem reliably, which a single greedy answer does not measure. We release code, tests and the ablation bundles.

1 Introduction

The challenge label marks a problem–model pair as robust when the model's accuracy holds across every perturbation applied by the organizers and non-robust when at least one perturbation clearly degrades it. The exact criterion is not published. Most interpretability methods would learn a proxy for this label from internal signals. We take the opposite route as a baseline: if the label is about behaviour under perturbation, measure that behaviour on the evaluated model itself, with perturbations we can generate offline. This gives a reference point for how much of the task is solvable without any access to internals, and its failure modes point at what a white-box method has to add.

2 Method

Perturbations. For each problem we build up to three variants that never touch mathematical tokens (digits, LaTeX commands, operators): (i) *typos*: swap two inner letters in every fourth plain word of length ≥ 5; (ii) *distract*: append one fixed irrelevant sentence with a number in it; (iii) *rename*: replace common character names when they appear (no-op otherwise). Organizer perturbations that need a generator model (domain transfer, rephrasing, expert edits) are not reproduced.

Answers. The original and each variant are answered by the evaluated checkpoint with greedy decoding, batched and sorted by length, loaded offline with `local_files_only`. The final answer

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

is the last `\boxed{}` after the reasoning, else the last number after `</think>`; truncated reasoning yields no answer.

Decision. A case is robust iff the original produced an answer and every variant's answer equals it. No answer, a time-out or a load failure yields `False`, the majority class in every public sample (robust share 0.32–0.375).

Budget. The 3600 s limit covers all models and problems. A global budget of 3300 s is split evenly across the remaining `are_robust` calls and passed to `generate(max_time=...)` per batch, so every call returns and coverage stays at 1.0.

3 Experimental setup

Main Track models: Qwen3.5-4B (thinking), Skywork-OR1-Math-7B, Olmo-3-7B-Think, DeepSeek-R1-0528-Qwen3-8B and gpt-oss-120b (low/medium/high effort); the Small Model Track drops gpt-oss-120b. We submitted the full method to both tracks and, on the Small track, three ablations built from the same file by changing one constant (`PC_PERTURBATIONS`): typos only, distractor only, and `none` (always `False`).

4 Results

Track	Variant	Codabench ID	Accuracy	Coverage
Small	full (typo+distract+rename)	1019144	0.571	1.00
Main	full (typo+distract+rename)	1019467	0.619	1.00
Small	always `False`	1019672	0.500	1.00
Small	typos only	–	pending	–
Small	distractor only	–	pending	–

Table 1: Validation leaderboard results. Pending rows are filled from the Codabench API before the report deadline.

5 Analysis of the public sample

The 28-row public sample (seven frontier models, AIMO-2025 reference problems) carries the per-case statistics behind the label. Two observations shape how this method should be read.

Base accuracy is a near-necessary condition. None of the 11 cases with base accuracy below 1.0 is robust; 9 of the 17 cases with base accuracy 1.0 are. A model that is only sometimes right on the original problem is almost always labelled non-robust, because any perturbation that costs one more sample counts as a clear degradation. Our method compares a single greedy answer per prompt, so it cannot see this: a model that is right 6 times out of 10 and greedy-stable across our three edits is called robust and is wrong.

Robustness is mostly a model property. Label means per model range from 0.0 (gpt-oss-120b, Gemini, Qwen3-8B) to 1.0 (GPT-5.2, GLM-5.1). This suggests that a per-model prior, estimated cheaply from the model's own sampling variance, carries much of the signal.

Implication. The natural extension, which we leave as the next step, replaces the greedy agreement test with a sampled estimate of base accuracy on the original problem (temperature 1.0, top-k 40, top-p 0.95, as in the organizers' generation) and requires both high self-agreement and stability under perturbation.

6 Conclusion

Perturbation self-consistency is a cheap, label-free baseline that runs within budget on every evaluated model, and on the validation leaderboard it beats the always-`False` predictor by 7 points on the Small track (0.571 against 0.500), with 0.619 on the Main track. The margin is small, and the public sample explains why it is not larger: the label is dominated by base-problem reliability, which greedy agreement does not measure. We offer the method and this negative result as a calibration point for black-box approaches.

Reproducibility

Code (MIT): `https://github.com/DesignMaster-Solutions/aimo-perturb-consistency`. The submission bundle is `solution.py` alone (plus an empty `small.txt` for the Small track); ablation bundles are produced by `./ablate.sh typo distract none`. No training data and no training codebase are used. Tests: `uvx pytest -q tests`.

AI usage statement

The method, code, tests and this report were developed with the help of an AI coding assistant (Claude) under the author's direction; all numbers are read from the Codabench API and the public sample.

Limitations

Our perturbation set covers three of the organizers' seven perturbation families. Greedy decoding differs from the sampled generation used to build the labels. The validation leaderboard has few cases per track, so differences of one or two cases move accuracy by several points.